Rapport d'architecture et de validation

01 Objet et état de la livraison

Contexte

RESULTATS-BF est une plateforme pédagogique de consultation de résultats d'examen pendant un pic de demandes. L'architecture doit répartir les requêtes, supporter la panne d'un serveur applicatif, répliquer les données et être reconstruite automatiquement. L'évaluation porte sur l'infrastructure, la reproductibilité et les preuves mesurées.

Groupe et périmètre

Le projet concerne le groupe 10 de l'Université Joseph Ki-Zerbo, IBAM, Master 1. Le socle demandé est complété par deux extensions : le cache Redis E3 et le déploiement progressif E4. La redondance du répartiteur E1 et la supervision E2 ne font pas partie du périmètre choisi.

État réel

La campagne VirtualBox complète a été exécutée sur la machine physique le 18 octobre 2026. Les sorties JSON, CSV et ApacheBench sont conservées dans results. Le rapport distingue les mesures finales réussies des essais intermédiaires ayant permis de diagnostiquer puis corriger les défauts.

Travail humain restant

Les noms, contributions et commits des quatre étudiants ne sont pas inventés. Ils doivent être ajoutés par les membres concernés. L'adresse 192.168.56.0/24 reste provisoire, car le tableau du sujet s'arrête au groupe 8 et doit être confirmé par l'enseignant.

02 Exigences et traçabilité

Reproductibilité

R1 est validée : la reconstruction destroy puis up a recréé les sept VM en 1 042,10 secondes, sans commande manuelle dans les invités. R2 est validée selon F(F(x)) = F(x) : après convergence, la seconde exécution du provisionnement a produit les mêmes empreintes sur chaque nœud.

Évolutivité et santé

R3 est validée avec NB_APP=4 : l'inventaire et les serveurs HAProxy sont générés ensemble. V1 a distribué exactement cinq requêtes à chacun des quatre serveurs. R4 est validée : pendant V2, HAProxy a évincé app1 en 3,67 secondes après le premier échec observé et aucun échec n'a terminé après cinq secondes.

Données et confidentialité

R5 impose une réplication PostgreSQL en continu vers bdd2. R6 utilise une promotion contrôlée après arrêt vérifié de bdd1. R7 exclut les secrets, .env et les fichiers d'exécution de Git. R8 conserve le trafic des services sur le réseau privé et n'expose que le port applicatif de lb1.

Scénarios attendus

V1 à V5 sont obligatoires. V6 dépend de E1 et ne s'applique pas ici. E3 exige une comparaison chiffrée avec et sans cache. E4 exige une trace de requêtes concurrentes démontrant l'absence d'interruption pendant toute la mise à jour.

03 Topologie adresses et ressources

Plan d'adressage

Le réseau provisoire est 192.168.56.0/24. lb1 utilise .11, app1 à app4 utilisent .21 à .24, bdd1 utilise .31 et bdd2 .32. Les adresses sont produites par une seule source d'inventaire. Le contrôle préalable recherche les conflits avant tout démarrage.

Chemin du trafic

Le port 127.0.0.1:8080 de l'hôte mène à HAProxy sur lb1, puis aux applications sur le port 8000 et à PostgreSQL sur le port 5432. La réplication relie bdd1 à bdd2. Redis est joignable uniquement par les applications sur le réseau privé.

Budget mémoire

Le socle consomme 3 072 Mio : 768 Mio pour chaque base, 512 Mio pour app1, app2 et lb1. Avec quatre applications, le total atteint 4 096 Mio. Chaque VM reçoit un processeur virtuel. Toutes les VM partagent néanmoins le processeur, le disque et la mémoire de la même machine physique.

Limites

lb1 reste un point unique de panne. Les extensions retenues ne couvrent pas la redondance du répartiteur. L'arrêt de la machine hôte arrête l'ensemble du laboratoire. Les ressources disponibles sur l'hôte doivent être contrôlées avant un démarrage complet.

04 Provisionnement reproductible

Source unique

config/topology.rb valide le nombre d'applications et le sous-réseau, définit les VM, puis enregistre un inventaire JSON local. Le provisionnement Python reçoit exactement cet inventaire. HAProxy est généré depuis la même liste d'applications, ce qui évite une configuration manuelle divergente.

Installation

Toutes les VM utilisent bento/ubuntu-24.04 version 202502.21.0. Cette image a été téléchargée et utilisée avec succès par VirtualBox. bdd1 fournit apt-cacher-ng aux VM suivantes quand il est accessible. Les paquets sont regroupés par rôle et ne sont pas réinstallés inutilement.

Idempotence

Les fichiers ne sont remplacés que lorsque leur contenu change. Les insertions de données utilisent les conflits de clé primaire. Le marqueur d'initialisation empêche de refaire la copie de base de la réplique. Lors du test R2, les empreintes après la première et la seconde application étaient identiques sur les sept nœuds.

Bornes de récupération

Une promotion est conservée et empêche le provisionnement ordinaire de redémarrer l'ancienne base principale. Le retour à la topologie d'origine est une reconstruction complète et destructive de ce laboratoire jetable. Les changements de données de test sont alors supprimés.

05 Application sans état et modèle de données

Contrat HTTP

GET /info renvoie le nom du serveur et la version. GET /sante exécute SELECT 1 sur une base accessible en écriture et renvoie 503 en cas d'échec. GET /resultat exige un matricule à six chiffres. Une entrée invalide renvoie 400, un candidat absent 404 et une base indisponible 503.

Modèle

La table resultats contient matricule comme clé primaire texte, nom, serie, moyenne au format numérique et admis comme booléen. generate_series crée exactement 50 000 lignes fictives et déterministes, identifiées de 000001 à 050000. Les exécutions suivantes préservent les lignes existantes.

État

Les serveurs applicatifs ne conservent ni session, ni fichier métier, ni donnée autoritaire. PostgreSQL possède les résultats durables. Redis est un cache externe facultatif. Une VM applicative peut donc être supprimée et recréée sans perte de données.

Exploitation

Gunicorn exécute l'API sous un compte non privilégié. Les connexions à la base sont fermées après usage et les délais d'attente bornent les pannes ordinaires. Le code du service reste sous la limite de cent lignes. Les requêtes SQL utilisent des paramètres.

06 Réplication et promotion PostgreSQL

Réplication

PostgreSQL 16 sur bdd1 conserve les journaux nécessaires et utilise un emplacement physique pour bdd2. Un rôle dédié à la réplication n'est autorisé que depuis l'adresse privée de bdd2. pg_basebackup initialise la réplique qui fonctionne ensuite en mode récupération.

Procédure de panne

Il faut arrêter bdd1 puis vérifier que la VM est réellement hors tension. Le test a observé un HTTP 503 contrôlé. La commande de promotion refuse une ancienne base encore active, promeut bdd2, place son adresse en tête sur chaque application, redémarre les services et attend le retour de la santé publique.

Mesure

La première tentative a révélé que les applications essayaient trop longtemps l'ancienne base. Après correction de l'ordre DB_HOSTS, la reprise promotion plus santé a demandé 22,59 secondes. L'intervalle repris mesuré est de 22,66 secondes. Il exclut la durée d'arrêt antérieure et est étiqueté ainsi dans la preuve.

Limites de données

La réplication est asynchrone : une écriture très récente peut ne pas atteindre la réplique avant une panne. L'ancienne base principale doit rester arrêtée après promotion afin d'éviter deux bases en écriture. Aucun retour automatique ou consensus distribué n'est revendiqué.

07 Répartition et montée en charge

Distribution

HAProxy distribue les requêtes en tourniquet entre les applications générées. Après la reconstruction finale, V1 a donné cinq réponses à app1, app2, app3 et app4. L'écart relatif est donc nul, très inférieur au seuil de 20 pour cent.

Comportement en panne

Le contrôle /sante s'exécute chaque seconde et exige deux échecs avant le retrait. V2 a enregistré 2 939 requêtes pendant l'arrêt de app1, dont huit perdues. La période dégradée a duré 4,49 secondes et aucune erreur n'a terminé après cinq secondes.

Méthode d'extension

Le passage de deux à quatre applications utilise NB_APP=4. L'inventaire, les définitions des VM et les serveurs HAProxy évoluent ensemble. Pour réduire le nombre, il faut détruire explicitement app3 et app4 avant de revenir à NB_APP=2.

Résultat V3

Sans cache et avec ab -n 2000 -c 50, deux applications ont atteint 137,17 requêtes par seconde avec 364,52 ms de moyenne. Quatre applications ont atteint 121,28 requêtes par seconde avec 412,28 ms. Aucun échec n'a été observé. La baisse de 11,58 pour cent provient du partage du même hôte et constitue le résultat mesuré.

08 Extension E3 cache et invalidation

Choix

Redis fonctionne sur lb1 pour éviter une VM supplémentaire. Il écoute uniquement en local et sur le réseau privé, exige un mot de passe généré et utilise au maximum 64 Mio. Sa persistance est désactivée, car PostgreSQL reste la seule source durable.

Lecture

Une requête valide consulte d'abord une clé avec espace de noms. En cas de succès, la réponse JSON est renvoyée sans interroger PostgreSQL. En cas d'absence, l'application lit la base et écrit une entrée de trente secondes. Une panne Redis provoque un retour transparent vers PostgreSQL.

Cohérence

Un résultat en cache peut rester ancien pendant trente secondes après une modification. Une purge explicite est prévue après tout changement nécessitant une visibilité immédiate. /sante ignore toujours Redis afin que le cache ne masque jamais une panne de base au répartiteur.

Mesure

Avec quatre applications, le test sans cache a atteint 121,28 requêtes par seconde et 412,28 ms. Après purge puis préchauffage, Redis a atteint 1 271,20 requêtes par seconde et 39,33 ms, sans échec. Le débit est multiplié par 10,48 et la latence moyenne baisse d'environ 90,5 pour cent.

09 Extension E4 déploiement progressif

Procédure

Pour chaque application, l'outil vérifie qu'un autre serveur reste disponible, met la cible en drainage et attend la fin des sessions actives. Il passe ensuite la cible en maintenance, transfère le code, contrôle sa syntaxe, met à jour la version et redémarre le service.

Retour au service

Une vérification directe de /sante doit réussir. L'outil attend ensuite que HAProxy voie lui-même le serveur comme disponible avant de passer au suivant. L'absence d'état métier local rend cette rotation possible sans migration de session.

Gestion des erreurs

Une étape en échec arrête immédiatement le déploiement et laisse le serveur concerné isolé. Le retour automatique n'est pas implémenté. La version précédente doit être conservée dans Git et peut être redéployée après diagnostic.

Preuve

La validation finale couvre les 111,79 secondes du déploiement et cinq secondes supplémentaires. Elle a enregistré 17 155 requêtes en 118,78 secondes, sans aucun échec. Les contrôles /info ont observé les quatre applications et uniquement la version 2.

10 Campagne de validation mesurée

Protocole

Les tests ont été exécutés depuis la machine physique avec l'inventaire documenté et un état de cache connu. Les sondes enregistrent l'heure, la latence et le code de chaque requête. Les opérations produisent des fichiers JSON et ApacheBench conserve ses sorties complètes.

Tableau des résultats

V1 : réussi, 5/5/5/5. V2 : réussi, huit pertes et 4,49 secondes dégradées. V3 : mesuré, 137,17 requêtes par seconde à deux applications et 121,28 à quatre. V4 : réussi après correction, HTTP 503 puis 200 en 22,66 secondes reprises. V5 : réussi en 1 042,10 secondes. R2 et R5 : réussis. E3 : gain 10,48 fois. E4 : 17 155 requêtes sans échec.

Contrôles locaux

Vingt et un tests de l'application, de l'inventaire et des protections opérationnelles passent. Les syntaxes Python, Bash et Ruby ont été contrôlées. La reconstruction finale a ensuite vérifié les vrais services invités, puis V1 et R5 ont été rejoués avec succès.

Fichiers de preuve

Les preuves finales sont listées dans docs/VALIDATION.md avec leurs noms horodatés. Les essais infructueux restent dans results afin de montrer les causes identifiées et les corrections apportées. Ils ne remplacent pas les fichiers finaux indiqués dans la matrice.

11 Sécurité limites et disponibilité

Secrets

setup.py crée des identifiants aléatoires dans un fichier .env protégé. .env et .runtime sont ignorés par Git et exclus de l'archive. Chaque VM ne reçoit que les secrets nécessaires à son rôle. Les fichiers temporaires sont ensuite supprimés et les permissions sont limitées.

Réseau

Les services communiquent sur le sous-réseau privé. Seul lb1 publie le port applicatif 8080. Chaque VM possède également une redirection SSH limitée à la boucle locale pour l'administration Vagrant. Cette exception de gestion est déclarée et doit être confirmée si R8 est interprétée de manière stricte.

Incidents rencontrés

Deux anciens processus VirtualBox utilisaient encore les adresses .31 et .32. Les adresses MAC ont permis de les identifier puis un arrêt propre les a isolés. Le système de fichiers de app4 est devenu en lecture seule après une interruption ; cette VM sans état a été recréée.

Limites connues

Le laboratoire n'inclut pas TLS, la redondance de lb1, le basculement automatique par consensus ni une stratégie de sauvegarde durable. Ces limites sont hors des extensions choisies. La plateforme est destinée à une démonstration locale et reproductible.

12 Exploitation équipe et références

Démarrage

scripts/start.sh crée les secrets manquants, vérifie les prérequis, valide Vagrant, démarre le laboratoire puis contrôle la distribution. La reconstruction exige un indicateur destructif explicite et ne cible que les VM de ce projet. La VM Ubuntu préexistante n'est jamais une cible.

Contributions

Les quatre membres doivent prendre en charge le réseau, l'application, les données et la validation, puis réaliser chacun une contribution réelle hors de leur rôle principal. Le guide Git joint explique comment enregistrer ces changements avec les vraies identités sans fabriquer d'historique.

Démonstration

La présentation de vingt minutes doit couvrir l'architecture, la reconstruction de 17 minutes 22 secondes, les pannes et les performances. Chaque membre doit pouvoir expliquer la clôture de l'ancienne base, la cohérence limitée du cache, la méthode de déploiement et la baisse mesurée avec quatre applications.

Références

La source principale est le sujet RESULTATS-BF de quatorze pages. Les références techniques concernent Vagrant, PostgreSQL, HAProxy et les métadonnées officielles de l'image Bento. Les adresses complètes figurent dans docs/SOURCES.md et le fichier box-metadata.json est conservé.